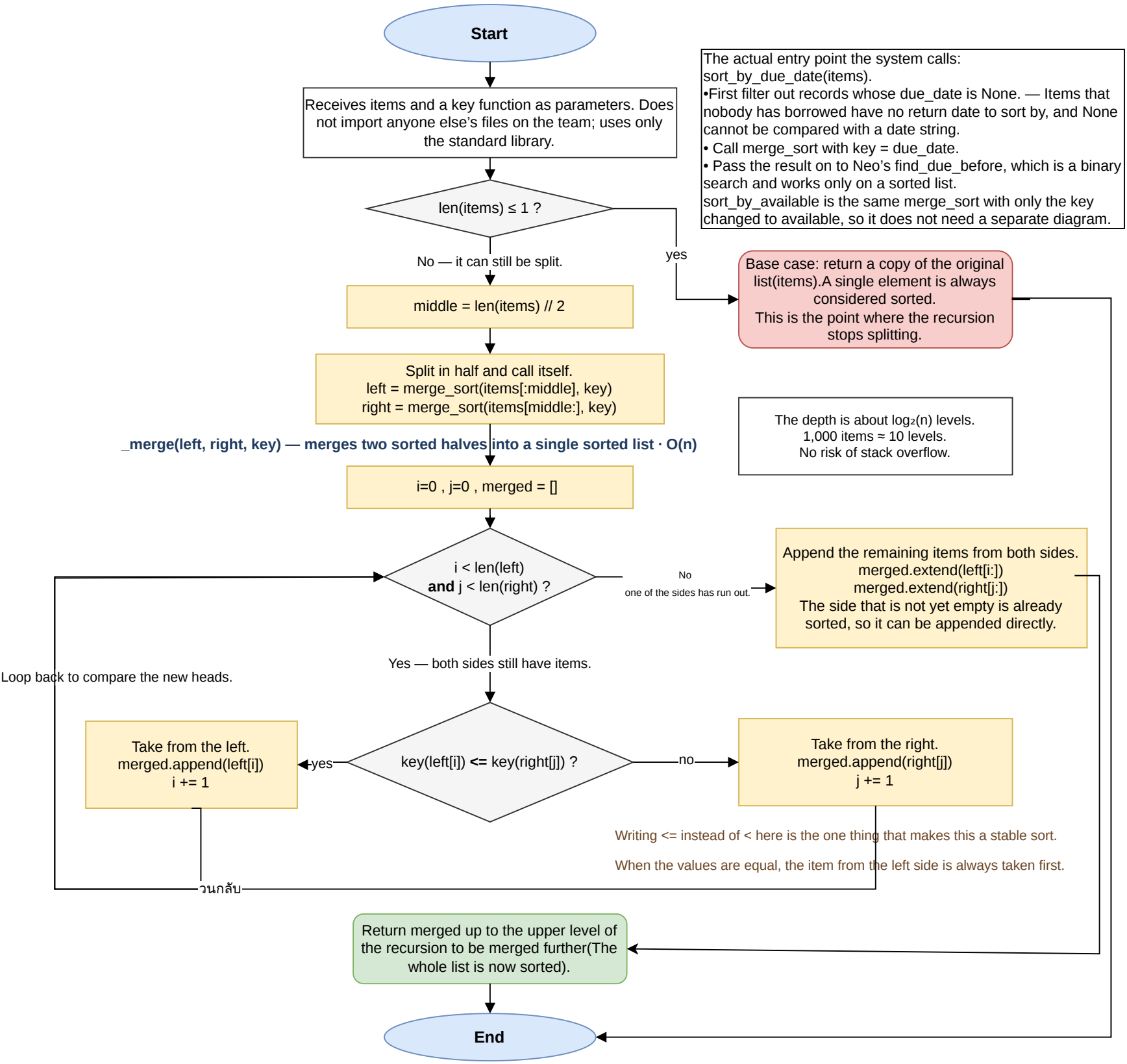
Flowchart 6 — Sorting: merge_sort (split in half → merge)

sort_group.py · The structure is recursive, splitting in half repeatedly until a single element remains. Unlike hash_table.py, which computes the position directly from the key, and unlike count_* in search.py, which finishes in a single pass over the data.



The actual entry point the system calls: sort_by_due_date(items).
•First filter out records whose due_date is None. — Items that nobody has borrowed have no return date to sort by, and None cannot be compared with a date string.
• Call merge_sort with key = due_date.
• Pass the result on to Neo's find_due_before, which is a binary search and works only on a sorted list.
sort_by_available is the same merge_sort with only the key changed to available, so it does not need a separate diagram.

The depth is about $\log_2(n)$ levels.
1,000 items ≈ 10 levels.
No risk of stack overflow.

Writing <= instead of < here is the one thing that makes this a stable sort.

When the values are equal, the item from the left side is always taken first.

Why splitting in half and merging is faster than swapping pair by pair.

- Splitting down to 1 gives $\log_2(n)$ levels, and merging at each level takes O(n), for a total of O(n log n).
- Bubble sort must compare every pair over and over, so it is $O(n^2)$:Measured on dataset D with 1,000 items, merge sort is 34.6 times faster than bubble sort.
- Writing <= instead of < when comparing the left side makes it a stable sort.So the original order of items with the same due_date is not shuffled for no reason.
- It always returns a new list and never modifies the original, because someone else may be holding the same list.This is the same approach as sort_by_code in search.py.
- The sorted list from this file is what find_due_before (Neo's binary search) uses directly.If the sorting is wrong, the binary search will cut the wrong half without raising any error.